Yggdrasil ERP — Server Architecture

Mimir Labs Technical Publication

Document: ML-WP-002 **Version:** 1.0
Author: Christopher Gaither **Date:** March 2026
Classification: Public

0.1 Executive Summary

Yggdrasil ERP is implemented as a deterministic application server designed to support governed enterprise operations rather than loosely coupled transactional sprawl. The server exposes HTTP and WebSocket interfaces to desktop, web, and mobile clients while coordinating authentication, authorization, state transitions, audit capture, event streaming, and module-level business logic.

The implementation is built in C++17 on Qt 6, but the more important architectural point is its operational posture: fail fast at startup, validate aggressively at boundaries, enforce permissions centrally, execute state changes atomically, and emit every consequential transition into an observable event stream. The result is a server designed not merely to process requests, but to preserve coherence across operational state.

This paper describes the Yggdrasil server as an architectural component of the wider Mimir Labs stack. It is not intended to be an exhaustive engineering reference. Its purpose is to explain how the server supports deterministic execution, tenant isolation, auditability, and governed system behavior.

0.2 1. Design Goals

The Yggdrasil server exists to operationalize the principles defined in the broader Yggdrasil architecture: canonical semantics, governed boundaries, observable integrations, and controlled system behavior. In practical terms, the server is designed around five goals.

First, it must behave deterministically under sustained enterprise workloads. Request handling,

memory behavior, service startup, and state transitions must be predictable enough to support manufacturing, finance, quality, and service operations.

Second, it must enforce security and tenancy centrally rather than relying on route-by-route discipline. Authentication, permission checks, and tenant resolution are handled as architectural concerns, not optional application conventions.

Third, it must treat state transitions as governed events. Business objects do not move from status to status through ad hoc updates; they move through explicit transition rules enforced by a state machine.

Fourth, it must produce an observable operational history. Significant mutations are logged, auditable, and available for downstream synchronization and compliance workflows.

Fifth, it must remain practical to deploy and operate. The system is designed to run headlessly in production, while still supporting operator-attended development and diagnostic workflows.

0.3 2. Technology Foundation

Yggdrasil is implemented in C++17 using Qt 6 for core runtime services, networking, SQL access, and WebSocket support. This choice was driven less by language preference than by operational requirements.

A native compiled runtime avoids garbage collection pauses and supports predictable resource behavior. Qt provides a mature cross-platform application model, robust SQL integration, thread management, and networking primitives without forcing the system into a heavyweight web framework. That combination is well suited to a server that must coordinate ERP workflows, route large numbers of typed API operations, and maintain real-time communication channels.

The server communicates with PostgreSQL through Qt's PostgreSQL driver and relies on structured configuration files for runtime settings. These implementation choices support portability across development and production environments while keeping the runtime model explicit and inspectable.

0.4 3. Startup and Runtime Posture

Yggdrasil follows a fail-fast startup model. Critical dependencies are initialized in a strict sequence, and the server does not begin accepting requests until the core runtime is coherent.

At startup, the server loads configuration, initializes structured logging, establishes database connectivity, verifies schema state, prepares in-memory services, initializes authentication and secrets handling, starts event infrastructure, and only then binds the HTTP interface and registers routes.

This ordering is deliberate. A partially initialized ERP server is more dangerous than a failed one because it creates ambiguous operational behavior.

The runtime supports both headless and attended modes. In production, it typically runs as a daemonized service with no display requirements. In development or operator-attended scenarios, the same server can expose a local status interface for diagnostics and monitoring. The architectural significance is not the GUI itself; it is that the same execution core supports both deployment and inspection without requiring a separate administrative control plane.

This startup discipline contributes directly to one of Yggdrasil's central promises: the server should either be operationally valid or unavailable, but never silently degraded.

0.5 4. Security Architecture

Security in Yggdrasil is layered. Authentication establishes identity, authorization constrains action, tenancy constrains scope, and middleware ensures these checks are applied consistently.

Passwords are stored using modern key-stretching and compared using constant-time techniques. Multi-factor authentication is supported through time-based one-time passwords, along with recovery mechanisms and progressive account lockout. Session management uses short-lived access tokens and longer-lived refresh tokens, transmitted through secure cookie controls to reduce browser-side exposure.

These implementation details matter, but the more important architectural property is centralization. Authentication is not reimplemented at the module level. The server treats identity as a shared system service that all routes and services depend on.

Authorization is likewise centralized. Every authenticated request is evaluated against a role hierarchy and module-level permissions before it reaches application logic. Unrecognized or insufficiently described routes fail closed. That posture reduces the risk that a newly added endpoint accidentally becomes an ungoverned bypass around the permission system.

The server also supports centralized multi-tenant authentication patterns, enabling deployments in which identity can be validated against a shared registry while local tenant context and role mappings remain enforceable within the application boundary.

0.6 5. Request Governance and Middleware

Every incoming request traverses a middleware pipeline before business logic executes. This is one of the most important architectural features of the server because it establishes the application boundary as a governed boundary.

The pipeline applies origin controls, request throttling, token validation, tenant resolution, and role

enforcement before a request reaches its target route. Responses then pass through serialization and audit-related handling before completion. The result is a consistent contract: every business operation enters the server through the same security and governance envelope.

Rate limiting protects both interactive and programmatic interfaces from accidental or deliberate abuse. Tenant resolution ensures that requests are scoped to the correct organizational boundary before data access begins. RBAC enforcement ensures that authenticated identity does not imply unrestricted capability.

This layered request model is significant beyond HTTP mechanics. It is the server-side embodiment of the broader Yggdrasil principle that boundaries must be governed, not assumed.

0.7 6. API and Module Architecture

The Yggdrasil server exposes a modular API surface spanning the operational domains expected of an ERP platform: customer and sales operations, purchasing, manufacturing, warehouse activity, finance, quality, service, workflow, forms, integration, administration, and related support capabilities. Internally, these capabilities are organized as route modules registered at startup.

The architectural value of this modular route design is separation without fragmentation. Each domain can evolve as a self-contained unit, but all modules still inherit the same authentication model, permission enforcement, response conventions, audit patterns, and tenant scoping behavior. In other words, Yggdrasil does not permit each module to become its own miniature application with its own rules.

Common route helpers standardize recurring patterns such as paginated listing, single-record retrieval, mutation logging, and identity resolution. This reduces implementation drift across modules and makes the API surface more predictable for both client applications and future maintainers.

The server also publishes machine-readable API documentation. That is not merely a developer convenience. In a governed system, explicit interface contracts are part of the architecture itself. They help constrain client behavior, clarify payload structure, and support downstream tooling such as validation, testing, and integration review.

0.8 7. State Machine Execution Model

The most important service in the Yggdrasil server is the state machine engine. Rather than allowing business objects to move through lifecycles by arbitrary field mutation, the server defines valid transitions for each governed entity type and rejects transitions that fall outside the allowed graph.

This is a major architectural distinction from conventional ERP implementations, where status fields often degrade into loosely governed metadata. In Yggdrasil, state is not descriptive decoration; it

is part of the execution model.

When a valid transition occurs, the server performs the update atomically together with audit capture and event emission. That means the operational record, the audit history, and the real-time event stream remain synchronized. A state change is therefore not just a database update. It is a governed transition with traceable consequences.

This model becomes especially important in domains such as manufacturing, approvals, purchasing, finance, quality, and service, where sequence matters and downstream logic depends on trustworthy lifecycle status.

0.9 8. Service Layer and Supporting Runtime

Around the state machine sits a supporting runtime of shared services.

The cache layer reduces redundant reads while preserving coherence through write-triggered invalidation. Metrics services expose route-level and performance data for operational monitoring. Notification services deliver user-facing events through real-time channels and asynchronous fallbacks. Secrets management protects sensitive configuration and credential material at the application layer.

These are familiar service categories, but in Yggdrasil they are intentionally subordinate to the governed execution model. They exist to support deterministic operations rather than to create a loosely coupled microservice sprawl. The server remains a single coordinated application boundary in which operational behavior is centralized and inspectable.

0.10 9. Data Access and Persistence Model

The server's persistence layer is built around PostgreSQL and a repository abstraction that standardizes connection handling, prepared queries, transaction boundaries, and tenant-aware data access. Dynamic queries are constructed through safe parameterization rather than ad hoc SQL concatenation.

The database schema and migration system are treated as part of the operational contract of the server. Schema verification occurs during startup, and migrations are applied in a controlled, sequential manner. This helps ensure that runtime behavior and persistence structure do not drift apart across deployments.

The server's relationship to the database is therefore not that of an application casually reading and writing rows. It is a coordinated execution layer operating against a governed persistence model. That distinction becomes more important as the system is used in audit-sensitive or integration-heavy environments.

0.11 10. Real-Time Event Architecture

Yggdrasil includes a real-time event hub that exposes operational events over WebSocket channels and can relay them outward to message-oriented infrastructure. Events include state transitions, data changes, workflow activity, and notifications.

This event system serves two purposes.

Internally, it gives clients and supporting services immediate visibility into consequential operational changes. Externally, it creates a clean integration seam for synchronization, archival, or downstream processing without forcing external systems to poll the transactional API.

Events are tenant-scoped, authenticated, and emitted from governed application paths rather than from uncontrolled database triggers or best-effort side effects. That is an important architectural property. The event stream is not an approximation of system activity; it is a structured expression of validated system behavior.

A relay layer can forward these events to Kafka-compatible infrastructure for broader distribution, replay, or integration use cases. This provides a bridge between Yggdrasil's deterministic execution core and the wider synchronization patterns envisioned elsewhere in the Mimir Labs stack.

0.12 11. Operational Characteristics

The server is designed to be operable in production with a small and explicit control surface. Health endpoints report runtime and dependency status. Metrics endpoints expose counters and timings suitable for external monitoring. Logging is structured and rotation-aware. Shutdown behavior is ordered so that active requests, event buffers, and database connections are drained cleanly.

These operational properties matter because enterprise reliability depends as much on observability and failure handling as on feature scope. A deterministic ERP core cannot rely on informal runtime practices. It must expose enough structured behavior to be monitored, diagnosed, and trusted in live operations.

Deployment is similarly pragmatic. The current model assumes an internet-shielded host, TLS protection at the edge, containerized orchestration, and routine health checks. These are conventional choices, but they support an architectural theme that runs throughout Yggdrasil: reduce ambiguity, keep boundaries explicit, and make the runtime state observable.

0.13 12. Architectural Role in the Mimir Stack

The Yggdrasil server should not be understood as an isolated application tier. It is the execution core of a wider governance-first architecture.

Within that architecture, Mimisbrunnr defines canonical meaning, Ratatosk exposes semantic structure from existing environments, and Yggdrasil operationalizes governed enterprise state. The server is the mechanism by which those principles become executable. It receives requests, validates identity and scope, enforces transition rules, persists operational truth, and emits observable consequences.

That role is strategically important. Many enterprise systems can record transactions. Far fewer can maintain a governed and inspectable operational state while remaining compatible with synchronization, migration, and future semantic tooling. The Yggdrasil server is designed specifically for that role.

0.14 Conclusion

The Yggdrasil server is not merely a web application backend for an ERP interface. It is a deterministic execution boundary for governed enterprise operations.

Its architectural significance lies in the way it combines centralized security, explicit request governance, modular APIs, state-machine enforcement, structured audit capture, and observable event emission into a single coherent runtime. Those properties allow the server to support more than transactional throughput. They allow it to serve as the operational heart of a governance-native system.

As the broader Mimir Labs stack evolves, the server provides the practical bridge between canonical semantics and executable business state. It is the layer where governed meaning becomes governed operation.